

Semantica di P_0

Definizione 3.2

Un assegnamento B è una funzione:

$$B : \begin{array}{l} \text{variabili} \\ \text{proposizionali} \end{array} \rightarrow \{0, 1\}$$

Definizione 3.3

Un'assegnamento proposizionale B è esteso per induzione ad una valutazione $\bar{B}$ del linguaggio P_0 nel modo seguente

- $\bar{B}(p) = B(p) \quad \forall p$
- $\bar{B}(\alpha \rightarrow \beta) = \begin{cases} 0 & \text{se e solo se } \bar{B}(\alpha) = 1 \text{ e } \bar{B}(\beta) = 0 \\ 1 & \text{altrimenti} \end{cases}$
- $\bar{B}(\neg \alpha) = 1 - \bar{B}(\alpha)$

Definizione 3.4

Una fbf α di P_0 è detta essere:

- **tautologia**: $\forall B \Rightarrow \bar{B}(\alpha) = 1$
- **Soddisfacibile**: $\exists B \mid \bar{B}(\alpha) = 1$
- **Contraddittoria**: $\exists B \mid \bar{B}(\beta) = 1 \quad \forall \beta \in \Gamma$

Proposizione 3.13

Una fbf α di P_0 è una tautologia se e solo se $\neg \alpha$ è contraddittoria

Dimostrazione

Se utilizziamo le def 3.3 notiamo che $\bar{B}(\neg \alpha) = 1 - \bar{B}(\alpha) = 0$

Proposizione 3.14

Se α e $\alpha \rightarrow \beta$ sono tautologie $\Rightarrow B$ è una tautologia

$$\bar{B}(\alpha) = 1 \quad \bar{B}(\alpha \rightarrow \beta) = 1 \Rightarrow 1 \Rightarrow \bar{B}(\beta) = 1$$

Osservazione

$\bar{B}(\beta) = 1$ sempre se $\bar{B}(\alpha \rightarrow \beta) = 1$

Lemme 3.1

Dato un fbf α di P_0 , $\bar{B}(\alpha)$ dipende solo dal valore assegnato da B alle variabili proposizionali in α (perché B è una funzione)

Teorema

Dato fbf α di P_0 , è decidibile se α è una tautologia
 $\Rightarrow$

Dimostrazione

Si verifica facendo le tabelle di verità di α

Definizione 3.5 (conseguenze tautologiche)

Dato una fbf α di P_0 , e un insieme Γ di fbf di P_0 si dice che α è conseguenza tautologica di Γ se e solo se

per ogni assegnamento proposizionale B si ha che:

Se per ogni fbf $\beta \in \Gamma$ vale $\bar{B}(\beta) = 1$ allora $\bar{B}(\alpha) = 1$

$\forall \beta \in \Gamma \mid \bar{B}(\beta) = 1 \Rightarrow \bar{B}(\alpha) = 1$ e scriviamo $\Gamma \models \alpha$

Teorema di correttezza di P_0 (3.5)

Siano Γ un insieme di fbf e α una fbf di P_0

Se $\Gamma \vdash_{P_0} \alpha$ allora $\Gamma \models \alpha$

"Se Γ deduce α allora α è una conseguenza tautologica"

Corollario 3.3

P_0 è consistente

Proposizione 3.15

Sia Γ un insieme di fbf di P_0 , se Γ è soddisfacibile allora Γ è consistente

Lemme 3.3 (soddisfacibilità)

Dato Γ un insieme di fbf di P_0 , se Γ è consistente allora Γ è soddisfacibile

Γ soddisfacibile $\Leftrightarrow \Gamma$ consistente

Teorema 3.6 (di completezza)

Sia Γ un'insieme di fbf di P_0 ed una fbf di P_0
de $\Gamma \models d$ se e solo se $\Gamma \vdash_{P_0} d$

Verso opposto del teorema di correttezza

Quindi teoremi 3.5 + 3.6 otteniamo il teorema di
correttezza e completezza

Corollario

Dato una fbf d di P_0 , è decidibile il problema di
dire se d è un teorema di P_0 (Problema di decisione)

Questo è riconducibile al problema di dire se d
è una tautologia (che è a sua
volta decidibile)

Introduzione di modelli Computazionali e nozioni fondamentali delle trasformazioni funzionali

Che significa computare?

trasformare un'informazione da una forma implicita ad una forma esplicita

forme implicite $\Rightarrow$ forme esplicite
 $(3+9)2 \Rightarrow 14$

Un modello computazionale è una specifica formalizzazione matematica del concetto di "computazione" e delle nozioni ad esso collegate come per esempio informazioni, trasformazioni, ecc...

I linguaggi di programmazione sono ispirati direttamente o indirettamente ad uno o più modelli computazionali

Linguaggi di tipo imperativi (c, pascal, fortran)

Sono quelli basati su modelli come la MT e la URM

URM = unlimited register machine è un modello computazionale basato su un numero illimitato di registri numerati; ognuno dei quali può contenere un numero naturale

Questo tipo di modello utilizza istruzioni semplici (incremento, decremento...) per manipolare i registri e definire algoritmi

L'URM è equivalente alle macchine di Turing in termini di potenza computazionale e viene usato per lo studio delle completezza e delle computabilità

Le nozioni fondamentali di tali linguaggi sono:

- Memoria
- variabile (qualcosa che si può leggere o modificare)
- assegnamento
- istruzione
- iterazione

Linguaggi di programmazione funzionali (Haskell, Scheme, Erlang)

tali linguaggi fanno riferimento al modello computazionale del λ -Calcolo.

Nozioni del tipo

- Espressioni
 - Variabili
 - Valutazione
 - Ricorsione
- entità
sconosciute
estrazione di un valore concreto

Linguaggi di programmazione logici (Prolog)

Questi si basano su alcune proprietà di sottosistemi delle logiche del primo ordine.

Il concetto di computazione coincide con il concetto di ricerca di una deduzione per una formula logica

Le proprietà della funzione

La funzione che restituisce l'indice di una stringa $x \in \Sigma^*$ è definita come:

$$\tilde{x} = \begin{cases} \tilde{E} & \text{se } x = E \\ \tilde{y}e & \text{se } x = ey \text{ con } e \in \Sigma \text{ e } y \in \Sigma^* \end{cases}$$

Questa definizione, oltre ad identificare univocamente una funzione sulle stringhe dell'alfabeto Σ , descrive implicitamente un metodo per calcolarla.

$$\tilde{cbol} = \tilde{bdc} = \tilde{dbe} = E d b e = d b e$$

$$\text{mv} [] = []$$

$$\text{mv} [e: y] = (\text{mv } y) ++ [e]$$

Quando chiediamo al Haskell di calcolare l'indice della stringa `cbol`

> mv "cbol"

> "dbe" (risposta)

Esercizio

non ho capito

Sistemi di equazioni differenziali:

$$f: \mathbb{R} \rightarrow \mathbb{R}$$

$$g: \mathbb{R} \rightarrow \mathbb{R}$$

$$f'' - 6g' = 6 \sin(x)$$

$$6g'' + e^2 f' = 6 \cos(x)$$

$$f(0) = 0, \quad f'(0) = 0, \quad g(0) = 1, \quad g'(0) = 1$$

Esempio per cui non tutte le definizioni di funzioni descrivono anche un procedimento per il loro calcolo

Una versione equivalente di `mv` del linguaggio Haskell:

`mv = \y → if (null y) then y
 else (mv (tail y)) ++ [head y]`

`null`: restituisce `true` se l'argomento è la lista vuota
 altrimenti

`tail`: prende come argomento e restituisce una lista uguale a quella che abbiamo ma senza il primo elemento

`head`: prende una lista come argomento e restituisce il suo primo elemento

`++` prende due liste come argomenti e restituisce la loro concatenazione

`if... then... else` espressioni condizionali

"`\y →`" questa è una funzione che se prende in input un argomento `y` restituisce quello che c'è dopo la freccia

"`mv =`" `mv` è il nome

`square x = x * x` forma compatta per definire il quadrato di `x` in Haskell

`square x → x * x`

Definiamo il fattoriale:

`fact 0 = 1`

`fact n = n * (fact (n-1))`

forma compatta

`fact = \n → if (n == 0) then 1 else n * (fact (n-1))`

Definiamo come definire una funzione in un linguaggio di prog funzionale

1) Costruire un'espressione matematica:

variabili, costanti, funt predefinite e inserire nomi di funz già pre-esistenti $\Rightarrow$ Applicazione funzionale

2) Costruire una funzione anonima

definire tale espressione chiamata corpo della funzione
Identifichiamo una variabile (che chiamiamo parametro formale) e utilizziamo l'operatore funzione

$\lambda \rightarrow$ operatore di astrazione funzionale

3) Associare la funzione anonima ad un nome utilizzando l'operatore =

Computazione nei linguaggi funzionali:

> non "eval"

Indichiamo con $\rightsquigarrow A$ $\rightsquigarrow B$ $\rightsquigarrow C$ differenti passi di Computazione

- Primo passo computazionale

$\rightsquigarrow A$ sostituire un nome con l'espressione associata ad esso

- $\rightsquigarrow B$ sostituire il parametro attuale al posto del parametro formale

- $\rightsquigarrow C$ calcolo di una funzione predefinita e cui abbiamo applicato un valore esplicito

"(f x)"

$$f(x) = x + 3$$

↓
parametro funzione f

$$f(x) = x + 3$$

ogni passo computazionale $\rightarrow$ trasformare una sott'espressione

Popime 8 studiare codice funzione mv

Vedere come sono correlati $\begin{matrix} \sim A \\ \sim B \\ \sim C \end{matrix}$ per fare queste cose

Linguaggio imperativo: scheme

```
Boolean esen := true; // variabile globale
:
PROCEDURE calcolate (INTEGER value): Integer;
begin
    esen := not esen; // variabile globale dell'ambiente
    if esen
    then Return (value + 1)
    else Return (value + 2)
    end if
end
:
print (calcolate (6));
:
print (calcolate (6));
```

il valore restituito dipende dal numero di volte che viene eseguito il codice

Side effect: che è proprio dei linguaggi imperativi e non dei linguaggi funzionali

le due print sono identiche ma queste producono risultati diversi $\begin{matrix} 4 = (6+1) \\ 8 = (6+2) \end{matrix}$

non dipende dall'argomento che dà alla funzione
Questo valore dipende dalla variabile globale esen
la cui modifica rappresenta un side effect

Imbecille nella programmazione funzionale poiché le funzioni definite sono delle vere e proprie funzioni

motemotiche il valore di una sottoproposizione è sempre lo stesso indipendentemente da dove si trova e da quante volte viene valutata. Questa proprietà si chiama *referential transparency*.